

Why do we need Dependency Injection?

Without DI:

```
public class OrderService {  
    private PaymentService paymentService = new PaymentService();  
}
```

Problems:

- Tight coupling
- Hard to test

Difficult to change implementations

With DI:

```
public class OrderService {  
    private PaymentService paymentService;  
}
```

Benefits:

- Loose coupling
- Easy unit testing
- Better maintainability

How Spring Boot Implements Dependency Injection

Spring Boot uses an IoC (Inversion of Control) Container that:

- Creates beans
- Manages bean lifecycle
- Injects dependencies at runtime
- Bean Creation Annotations

Spring automatically creates beans using:

- @Component
- @Service
- @Repository
- @Controller
- @RestController

Example:

```
@Service
public class PaymentService {
    public void pay() {
        System.out.println("Payment done");
    }
}
```

Types of Dependency Injection in Spring Boot

1. Constructor Injection (Recommended)

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Why preferred?

1. Immutable dependencies
2. Easier unit testing
3. No @Autowired required (Spring 4.3+)

2. Field Injection (Not Recommended)

```
@Service
public class OrderService {

    @Autowired
    private PaymentService paymentService;
}
```

Drawbacks:

- Hard to test
- Hidden dependencies
- Uses reflection

3. Setter Injection (Optional Dependency)

```
@Service
public class OrderService {

    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Use this when the dependency is optional.

Multiple Implementations – @Qualifier

Interface:

```
public interface PaymentService { void pay(); }
```

Implementation 1:

```
@Service("upi")
public class UpiPaymentService implements PaymentService {
    public void pay() { }
}
```

Implementation 2:

```
@Service("card")
public class CardPaymentService implements PaymentService {
    public void pay() { }
}
```

Injection:

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(@Qualifier("upi") PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Using @Primary

```
@Primary
@Service
public class UpiPaymentService implements PaymentService {
}
```

Spring injects this implementation by default if no qualifier is specified.

Bean Lifecycle (Interview Favorite)

- Bean creation
- Dependency injection
- @PostConstruct
- Bean ready for use
- @PreDestroy

Real-World Spring Boot Example

```
@RestController
@RequiredArgsConstructor
public class OrderController {

    private final OrderService orderService;

    @PostMapping("/order")
    public String placeOrder() {
        orderService.placeOrder();
        return "Order placed";
    }
}
```

Using Lombok:

```
@RequiredArgsConstructor
```

Automatically creates a constructor for final fields.

Interview One-Liner

Dependency Injection is a design pattern where Spring manages object creation and injects dependencies at runtime, ensuring loose coupling, testability, and maintainability.